



ДИСЦИПЛИНА

Разработка приложений на языке Котлин

(полное наименование дисциплины без сокращений)

ИНСТИТУТ

информационных технологий

КАФЕДРА

информационных технологий в атомной энергетике

полное наименование кафедры

ВИД УЧЕБНОГО
МАТЕРИАЛА

Лекция

(в соответствии с пп 1-11)

ПРЕПОДАВАТЕЛЬ

Золотухин Святослав Александрович

(фамилия, имя, отчество)

СЕМЕСТР

5 семестр 2025 – 2026 учебный год

(указать семестр обучения, учебный год)



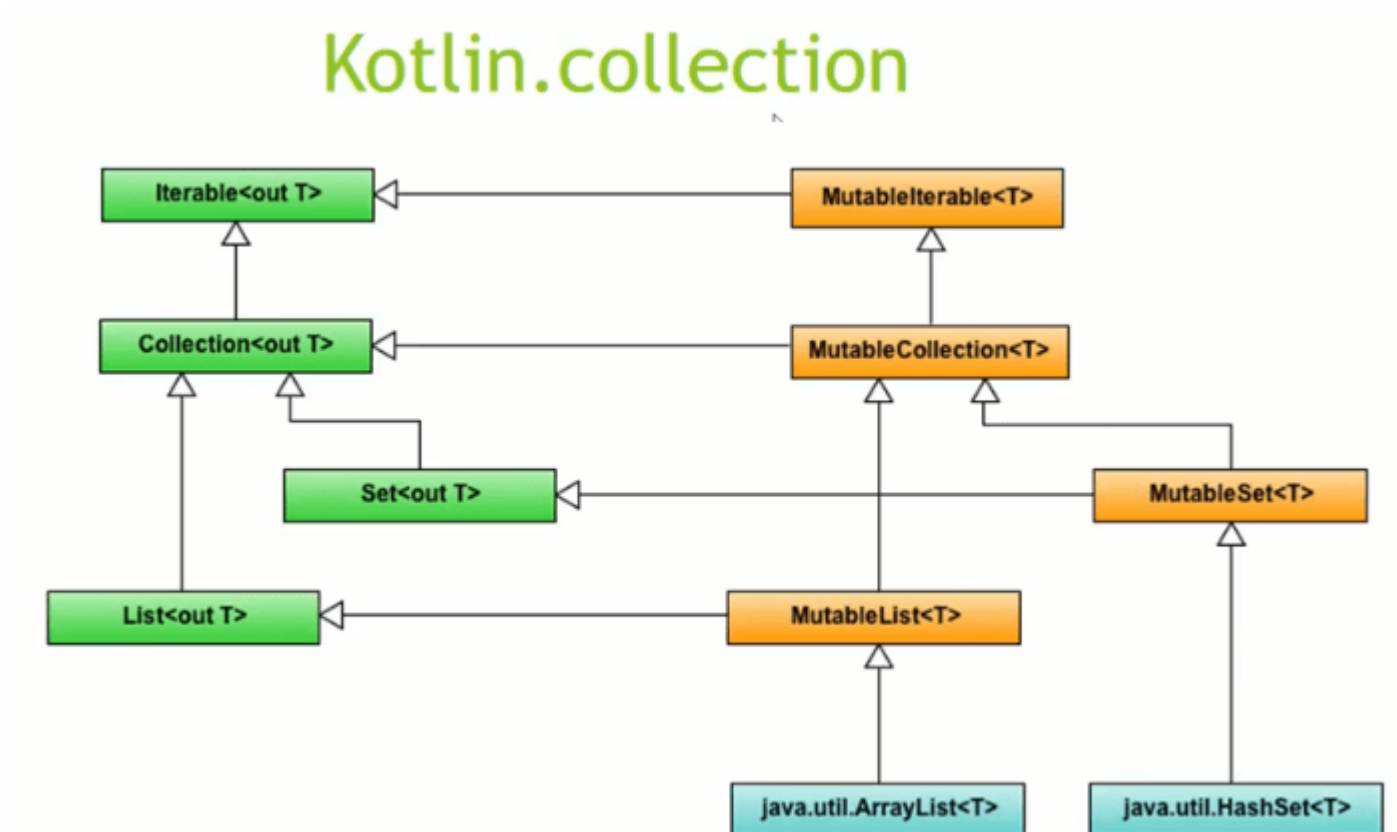
Лекция 7. Разработка приложений на языке Котлин

ФИО преподавателя: Золотухин Святослав Александрович

e-mail: zolotuhin@mirea.ru



Коллекции и последовательности





Коллекции и последовательности

Неизменяемые коллекции

Основным интерфейсом, который позволяет работать с коллекциями, является **kotlin.Collection**. Основные компоненты:

- **size**: возвращает количество элементов в коллекции
- **isEmpty()**: возвращает true, если коллекция пустая
- **contains(element)**: возвращает true, если коллекция содержит element
- **containsAll(collection)**: возвращает true, если коллекция содержит элементы коллекции collection



Коллекции и последовательности

Изменяемые коллекции

Все изменяемые коллекции реализуют интерфейс **MutableIterable**. Он представляет функцию итератора для перебора коллекции.

Для изменения данных в Kotlin также определен интерфейс **kotlin.MutableCollection**, который расширяет интерфейс **kotlin.Collection** и предоставляет методы для удаления и добавления элементов. В частности:

- **add(element)**: добавляет элемент
- **remove(element)**: удаляет элемент
- **addAll(elements)**: добавляет набор элементов
- **removeAll(elements)**: удаляет набор элементов
- **clear()**: удаляет все элементы из коллекции



Коллекции и последовательности

Операции с коллекциями

- **average(): Double**

возвращает среднее значение для числовой коллекции типов Byte, Int, Short, Long, Float, Double

- **contains(element: T): Boolean**

возвращает true, если коллекция содержит элемент element

- **count(): Int**

возвращает количество элементов в коллекции

.....



Коллекции и последовательности

List

Объект List<Int>

```
var numbers = listOf(1, 2, 3, 4, 5)
```

Объект List<String>

```
val humans = listOf("Andrew", "Alice", "Kate", "Antony", "Mary")
```

```
for(person in humans) println(person)
```

```
println(people) // [Andrew, Alice, Kate, Antony, Mary]
```



Коллекции и последовательности

Методы списков

Для получения элемента по индексу применяется метод `get(index)`

```
val humans = listOf("Andrew", "Alice", "Kate",  
"Antony", "Mary")
```

```
val first = humans.get(1)
```

```
print(first) // Alice
```




Коллекции и последовательности

Методы списков

Для получения элемента по индексу применяется метод `get(index)`

```
val humans = listOf("Andrew", "Alice", "Kate",  
"Antony", "Mary")
```

```
val first = humans.get(1)
```

```
print(first) // Alice
```

```
print(humans[0]) // Andrew
```



Коллекции и последовательности

Методы списков

Если индекс выходит за границы списка то при использовании **get()** и **[]** генерируется исключение.

Чтобы это избежать можно применить метод **getOrNull()**

```
val humans = listOf("Andrew", "Alice", "Kate",  
"Antony", "Mary")  
print(humans.getOrNull(7)) // null
```

Либо применить метод **getOrNull()**



Коллекции и последовательности

Получение части списка

Метод `subList()` возвращает часть списка и в качестве параметров принимает начальный и конечный индексы извлекаемых элементов

```
val humans = listOf("Andrew", "Alice", "Kate",  
"Antony", "Mary")  
print(humans.subList(1,4) # [Alice, Kate, Antony]
```



Коллекции и последовательности

Изменяемые списки

Изменяемые списки представлены интерфейсом **MutableList**. Данный интерфейс реализуется классом **ArrayList**.

Для создания таких списков можно использовать методы:

- **arrayListOf():** создает объект **ArrayList**
- **mutableListOf():** создает объект **MutableList**



Коллекции и последовательности

Изменяемые списки

Если необходимо добавлять или удалять элементы, то можно использовать методы **MutableList**:

add(index, element): добавляет элемент по индексу

add(element): добавляет элемент

addAll(collection): добавляет коллекцию элементов

remove(element): удаляет элемент

removeAt(index): удаляет элемент по индексу

clear(): удаляет все элементы коллекции



Коллекции и последовательности

Set

Интерфейс **Set** представляет неизменяемый (**immutable**) набор уникальных элементов и расширяет интерфейс **Collection**.

Для создания неизменяемого (**immutable**) набора используется функция **setOf()**.

```
val nums = setOf(1, 2, 4)           // объект Set<Int>
```

```
val humans = setOf("Andrew", "Kate", "Alice") // объект  
Set<String>
```

```
for(person in people) println(person)
```

```
println(people) // [Andrew, Kate, Alice]
```



Коллекции и последовательности

Методы Set для операций с множествами

- **union**: объединение множеств
- **intersect**: пересечение множеств
(возвращает элементы, которые есть в обоих множествах)
- **subtract**: вычитание множеств (возвращает элементы, которые есть в первом множестве, но отсутствуют во втором)



Коллекции и последовательности

Методы Set для операций с множествами

```
val humans = setOf("Andrew", "Dmitriy", "Artem", "Michael")
```

```
val employees = setOf("Andrew", "Dmitriy", "Alice", "Kate")
```

```
// объединение множеств
```

```
val all = people.union(employees)
```

```
// пересечение множеств
```

```
val common = people.intersect(employees)
```




Коллекции и последовательности

Методы Set для операций с множествами

// вычитание множеств

val different = people.subtract(employees)

println(all) // [Andrew, Dmitriy, Artem, Michael, Alice, Kate]

println(common) // [Andrew, Dmitriy]

println(different) // [Artem, Michael]



Коллекции и последовательности

Изменяемые коллекции (MutableSet)

```
val nums: MutableSet<Int> = mutableSetOf(30, 31, 32)
```

Интерфейс `MutableSet` реализуется следующими типами изменяемых наборов:

- **LinkedHashSet**: объединяет возможности хеш-таблицы и связанного списка. Создается с помощью функции `linkedSetOf()`.
- **HashSet**: представляет хеш-таблицу. Создается с помощью функции `hashSetOf()`.



Коллекции и последовательности

Map

Коллекция Map представляет коллекцию объектов, где каждый элемент имеет ключ и сопоставляемое с ним значение, при этом ключи являются уникальными.

```
val humans = mapOf(1 to "Andrew", 2 to "Vlad", 7 to "Kate")
```

Функция **mapOf** принимает набор элементов, каждый из которых с помощью оператора **to** сопоставляет ключ со значением.



Коллекции и последовательности

Перебор Map

```
val humans = mapOf(1 to "Andrew", 2 to "Vlad", 7 to "Kate")
```

```
for(person in people){  
    println("${person.key} - ${person.value}")  
}
```

```
println(humans)    // {1=Andrew, 2=Vlad, 7=Kate}
```

```
C:\Users\svt\.jdk\corretto-17.0.8.1\bin\java.exe
```

```
1 - Andrew
```

```
2 - Vlad
```

```
7 - Kate
```

```
{1=Andrew, 2=Vlad, 7=Kate}
```



Коллекции и последовательности

Обращение к элементам Map

```
val colors = mapOf("white" to "белый", "blue" to "синий",  
"green" to "зеленый")
```

```
val green = colors.get("green")
```

```
println(green) // синий
```

```
print(colors["blue"]) // синий
```

```
print(colors["purple"]) // null
```



Коллекции и последовательности

Обращение к элементам Map

```
val colors = mapOf("white" to "белый", "blue" to "синий",  
"green" to "зеленый")
```

```
val yellow = dictionary.getDefault("yellow", "Undefined")  
println(yellow) // Undefined
```

```
print(colors.getOrElse("yellow"){ "Not found" }) # Not Found
```



Коллекции и последовательности

Обращение к элементам Map

```
val colors = mapOf("white" to "белый", "blue" to "синий",  
"green" to "зеленый")
```

```
print(colors.values) //[белый, синий, зеленый]
```

```
print(colors.keys) // [white, blue, green]
```

Также, с помощью методов **containsKey()** и **containsValue()** можно проверить наличие в Map определенного ключа и значения



Коллекции и последовательности

MutableMap

Изменяемые коллекции представлены интерфейсом `MutableMap`, который расширяет интерфейс `Map`

```
val humans = mutableMapOf(1 to "Andrew", 2 to "Vlad", 7 to "Kate")
```

Интерфейс **MutableMap** реализуется рядом коллекций:

- `HashMap`: простейшая реализация интерфейса `MutableMap`, не гарантирует порядок элементов в коллекции. Создается функцией `hashMapOf()`
- `LinkedHashMap`: представляет комбинацию `HashMap` и связанного списка, создается функцией `linkedMapOf()`



Коллекции и последовательности

MutableMap

Для изменения Map можно применять ряд методов:

put(key: K, value: V): добавляет элемент с ключом key и значением value

putAll(): добавляет набор объектов типа Pair<K, V>. В качестве такого набора могут выступать объекты Iterable, Sequence и Array

set(key: K, value: V): устанавливает для элемента с ключом key значение value

remove(key: K): V?: удаляет элемент с ключом key. Если удаление успешно произведено, то возвращается значение удаленного элемента, иначе возвращается null.



Коллекции и последовательности

MutableMap

Добавление данных

```
val humans = mutableMapOf(1 to "Andrew", 2 to "Vlad", 7 to  
"Kate")
```

```
humans.put(10, "Mike")
```

```
println(humans)    // {1= Andrew, 2= Vlad, 7= Kate, 10=Mike}
```



Коллекции и последовательности

MutableMap

Изменение данных

```
val humans = mutableMapOf(1 to "Andrew", 2 to "Vlad", 7 to  
"Kate")
```

```
humans.set(2, "Vladislav")
```

```
println(humans)    // {1= Andrew, 2= Vladislav, 7= Kate}
```



Коллекции и последовательности

MutableMap

Удаление данных

```
val humans = mutableMapOf(1 to "Andrew", 2 to "Vlad", 7 to  
"Kate")
```

```
humans.remove(2)
```

```
println(humans)    // {1= Andrew, 7= Kate}
```



Последовательности

Создание последовательности, функция `sequenceOf`

```
val humans = mutableMapOf(1 to "Andrew", 2 to "Vlad", 7 to  
"Kate")
```

```
println(humans.joinToString()) // Andrew, Vlad, Kate
```

Метод `asSequence` коллекций

```
val employees = listOf("Michael", "Antony", "Jane") // объект  
List<String>
```

```
val people = employees.asSequence() //тип  
Sequence<String>
```

```
println(people.joinToString()) // Michael, Antony, Jane
```



Последовательности

Создание последовательности, функция `generateSequence`

```
var number = 0
```

```
val numbers = generateSequence{ number += 2; number}
```

```
println(numbers.take(5).joinToString()) // получаем первые 5  
элементов последовательности - 2, 4, 6, 8, 10
```

```
var number = 0
```

```
val numbers = generateSequence{ number += 2; if(number > 8)  
null else number}
```

```
println(numbers.joinToString()) // 2, 4, 6, 8
```



Спасибо за внимание!